

Project 2 : Photo Editing Service

Assignment 7: Documentation

Dated: 07.12.2025

Contents

1	Scope and Goal of the Project	3
2	Backend Implementation	3
2.1	Project Layout	3
2.2	Core Settings (photo_editor_backend/settings.py)	4
2.2.1	Settings Overview	4
2.3	Custom User Model (accounts/models.py)	5
2.3.1	Model Definition	5
2.3.2	Explanation	5
2.4	Authentication Flow	5
2.4.1	Registration (/api/auth/register/)	5
2.4.2	JWT Login (/api/auth/login/)	6
2.4.3	Google OAuth (/api/auth/google/)	6
2.4.4	Token Refresh (/api/auth/token/refresh/)	7
2.4.5	Profile Management (/api/auth/profile/)	7
2.5	Gallery Domain (gallery app)	7
2.5.1	Models (gallery/models.py)	7
2.5.2	Serializers (gallery/serializers.py)	8
2.5.3	ViewSets (gallery/views.py)	9
2.6	How Components Interact (Runtime Flow)	10
2.6.1	General Request Flow	10
2.6.2	Media File Handling	10
2.6.3	Share Flow	11
2.7	Security & Production Checklist	12
3	Frontend Implementation	12
3.1	Project Architecture	12
3.1.1	Technology Stack	12
3.1.2	Directory Structure	13
3.2	Authentication System (AuthContext.jsx)	13
3.2.1	State Variables	13
3.2.2	Key Functions	14
3.2.3	Axios Interceptor (api/client.js)	14
3.3	Core Pages & Implementation Details	14
3.3.1	Dashboard (pages/Dashboard.jsx)	14
3.3.2	Photo Editor (pages/Editor.jsx)	15

3.3.3	Sharing (pages/Share.jsx)	15
3.3.4	Google Auth (pages/GoogleCallback.jsx)	15
4	Cloud Deployment and Infrastructure	16
4.1	Overview	16
4.2	Infrastructure Provisioning	16
4.2.1	Database Layer (Amazon RDS)	16
4.2.2	Object Storage Layer (Amazon S3)	16
4.2.3	Compute Layer (Amazon EC2)	17
4.3	Deployment Procedure	17
4.3.1	Containerization and Registry (Local Environment)	17
4.3.2	Server Configuration	18
4.3.3	Orchestration Setup	18
4.3.4	Execution and Verification	18
4.4	Future Enhancements	19
5	Future Improvements	19
6	References	19
7	Declarations	19



Figure 1: Dashboard

1 Scope and Goal of the Project

The primary goal of this project is to develop a secure, reliable, and scalable cloud-based photo editing and management service that enables users to store, organize, modify, and share their digital images with ease. We call this: **BugEdits**. The system aims to provide users with a seamless experience starting from account creation and authentication, through traditional email/password methods or OAuth providers and extending to full photo lifecycle management, including upload, editing, organization, and collaboration features. The scope of the project encompasses the implementation of robust user authentication, private-by-default photo storage, and intuitive tools for album creation and organization. Users will be able to upload multiple photos, store them securely in a cloud bucket (AWS S3) and perform essential editing operations including cropping, rotating, , and adjusting visual parameters like brightness and contrast etc.

2 Backend Implementation

2.1 Project Layout

```
photo_editor_backend/           # Django project root
manage.py                       # Django CLI entry point
photo_editor_backend/           # Project settings package
    __init__.py
    asgi.py
    wsgi.py
    settings.py                 # Global configuration (DB, auth, media, etc.)
    urls.py                     # Root URL dispatcher
accounts/                       # User & authentication app
    __init__.py
    admin.py
    apps.py
    models.py                   # Custom User model (extends AbstractUser)
    serializers.py              # UserSerializer (password handling, profile picture)
    views.py                    # RegisterView, GoogleLoginView, UserProfileView
```

```

    urls.py                # /api/auth/* endpoints
gallery/                  # Photo-gallery domain app
    __init__.py
    admin.py
    apps.py
    models.py              # Album, Photo, Share, PhotoEditLog
    serializers.py         # Serializers for each model + custom fields
    views.py               # AlbumViewSet, PhotoViewSet, ShareViewSet
    urls.py                # Router registration for gallery API
db.sqlite3                # Default SQLite DB (development)

```

2.2 Core Settings (photo_editor_backend/settings.py)

2.2.1 Settings Overview

- **BASE_DIR**: `Path(__file__).resolve().parent.parent`
Base directory for all relative paths.
- **SECRET_KEY**: Auto-generated string.
Must be kept confidential; replace with an environment variable in production.
- **DEBUG**: `True` (development).
Enables detailed error pages and static/media file serving.
- **ALLOWED_HOSTS**: `[]`. Populate with domain names during deployment.
- **INSTALLED_APPS**: Core Django apps + `rest_framework`, `django_filters`, `corsheaders`, `storages`, `accounts`, `gallery`. Registers all Django and project-specific components.
- **MIDDLEWARE**: Includes `corsheaders.middleware.CorsMiddleware` placed before `CommonMiddleware` to enable CORS.
- **ROOT_URLCONF**: `'photo_editor_backend.urls'`. Points to the root URL dispatcher.
- **DATABASES**:

```

{
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3'
    }
}

```

Uses SQLite for development; replace with PostgreSQL or MySQL as needed.

- **AUTH_USER_MODEL**: `'accounts.User'`. Tells Django to use the custom user model defined in the accounts app.
- **REST_FRAMEWORK**:

```
{
    'DEFAULT_AUTHENTICATION_CLASSES': (
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    )
}
```

All DRF endpoints require JWT authentication unless explicitly overridden.

- **CORS_ALLOW_ALL_ORIGINS:** True (development). Allows requests from any domain; in production replace with whitelisted origins.
- **MEDIA_URL / MEDIA_ROOT:** Media uploaded by users (photos, profile images) is stored under /media/ and BASE_DIR / 'media'.
- **STATIC_URL:** '/static/'. Static assets for Django's admin UI.
- **GOOGLE_OAUTH2_CLIENT_ID / GOOGLE_OAUTH2_CLIENT_SECRET:** Values from Google Cloud, used by `GoogleLoginView` to exchange authorization codes for tokens.
- **AUTH_PASSWORD_VALIDATORS:** Default Django validators ensuring strong password rules.
- **LANGUAGE_CODE, TIME_ZONE, USE_I18N, USE_TZ:** Set respectively to 'en-us', 'UTC', True, True.

2.3 Custom User Model (accounts/models.py)

2.3.1 Model Definition

```
class User(AbstractUser):
    bio = models.TextField(blank=True)
    profile_picture = models.ImageField(
        upload_to='profile_pics/',
        null=True,
        blank=True
    )
```

2.3.2 Explanation

- The model extends `AbstractUser`, preserving default fields such as username, email, password, first/last names.
- The `bio` field stores optional descriptive text about the user.
- The `profile_picture` field stores an uploaded image; both `null=True` and `blank=True` allow the field to be omitted.
- Because `AUTH_USER_MODEL` is set to 'accounts.User', all foreign keys referencing the user (e.g., in `gallery/models.py`) automatically use this custom model.

2.4 Authentication Flow

2.4.1 Registration (/api/auth/register/)

View: `RegisterView` (subclass of `generics.CreateAPIView`).

Serializer: `UserSerializer`, which defines a custom `create()` method that calls:

```
User.objects.create_user(**validated_data)
```

This ensures that Django automatically hashes the password using `set_password()`.

Process:

1. The frontend sends a POST request with JSON:

```
{
    username,
    email,
    password
}
```

2. Django REST Framework validates the fields using `UserSerializer`.
3. `create_user()` inserts a new row into `accounts_user`.
4. The response returns the serialized user data (excluding the password).

2.4.2 JWT Login (/api/auth/login/)

Authentication is handled by `TokenObtainPairView` from `rest_framework_simplejwt`.

- **Input:**

```
{ username, password }
```

- **Output:**

```
{ access: <jwt>, refresh: <jwt> }
```

- Tokens are signed using `SECRET_KEY`. The access token contains the `user_id` claim used by DRF's JWT authentication middleware.

2.4.3 Google OAuth (/api/auth/google/)

Flow Overview:

1. The frontend retrieves an authorization code from Google (via Google Sign-In UI).
2. It sends:

```
{ code }
```

to the Google OAuth endpoint.

The `GoogleLoginView` performs:

1. **Token Exchange** Sends a POST request to:
`https://oauth2.googleapis.com/token`

with:

- `client_id`,
- `client_secret`,
- `code`,
- `redirect_uri`,
- `grant_type=authorization_code`.

Receives: `access_token` (and optionally `id_token`).

2. **User Info Request** Sends:

```
GET https://www.googleapis.com/oauth2/v2/userinfo
Authorization: Bearer <access_token>
```

Retrieves the user's Google profile including email.

3. **User Matching** Looks up a **User** with the returned email.

- If no such user exists: returns HTTP 400 instructing the user to register first.
- If the user exists: proceeds to generate credentials.

4. **JWT Generation** Creates a refresh token:

```
RefreshToken.for_user(user)
```

Returns:

```
{ access: <jwt>, refresh: <jwt>, user: { id, username } }
```

5. **Client Storage** The frontend stores the access token and uses it for authenticated requests.

2.4.4 Token Refresh (/api/auth/token/refresh/)

- Accepts:

```
{ refresh: <token> }
```

- Returns:

```
{ access: <new_jwt> }
```

This enables the user to maintain long sessions without re-entering credentials.

2.4.5 Profile Management (/api/auth/profile/)

View: UserProfileView (subclass of RetrieveUpdateAPIView).

GET Request: Returns the serialized data of the currently authenticated user.

DELETE Request: Deletes the user account and all related data via Django's cascade rules.

2.5 Gallery Domain (gallery app)

2.5.1 Models (gallery/models.py)

Model	Primary Key	Important Fields	Relationships
Album	UUIDField (id)	title, description, is_public, created_at	owner → settings.AUTH_USER_MODEL (FK, related_name='albums')
Photo	UUIDField (id)	image (ImageField), title, uploaded_at	album (FK, nullable, related_name='photos'), owner (FK)
Share	UUIDField (token)	can_edit, redeemed, created_at	Either album xor photo (nullable FKs), shared_with (FK to User)
PhotoEditLog	Auto id	operation, details, timestamp, is_seen	photo (FK), editor (FK to User)

Validation Logic (`Share.clean()`)

```
if self.album and self.photo:
    raise ValidationError(
        'A share link can only be associated with either an album or a photo, not both'
    )
```

```
if not self.album and not self.photo:
    raise ValidationError(
        'A share link must be associated with either an album or a photo.'
```

This enforces that every share token references **exactly one** resource (album xor photo).

The `save()` method is overridden so that `clean()` is always executed before saving the model instance.

2.5.2 Serializers (`gallery/serializers.py`)

PhotoSerializer

- Standard fields.
- Adds a computed field `has_unseen_edits` (a DRF `SerializerMethodField`).
- Logic:
 - Returns `True` only if the requesting user is the owner **and** `obj.edit_logs.filter(is_seen=False)`.
 - All other users always receive `False`.

AlbumSerializer

- Nests photos using:

```
PhotoSerializer(read_only=True)
```
- Provides a complete album structure including embedded photo objects.

ShareSerializer

- Revalidates the XOR rule (same as `Share.clean()`).
- Accepts a simple `username` string and resolves it into a `User` instance for the `shared_with` FK.
- Makes sharing more user-friendly by avoiding raw user IDs.

SharedContentSerializer

- Used when a client redeems a share token.
- Returns a rich representation of the underlying object (album or photo).
- Includes the actual photo URL in a field such as `photo_image`.

ShareDetailSerializer

- Designed for managing outgoing shares (listing and editing permissions).
- Exposes:
 - `shared_with_username`
 - `shared_with_email`for UI display.

PhotoEditLogSerializer

- Provides a read-only field `editor_username`.
- Prevents exposing the entire User object while keeping attribution visible.

2.5.3 ViewSets (gallery/views.py)

All viewsets inherit from `ModelViewSet` and are registered using a DRF `DefaultRouter`.

AlbumViewSet

- `queryset = Album.objects.all()`
- `serializer_class = AlbumSerializer`
- Permissions:
 - Default: `IsAuthenticated`.
 - Additional restrictions (conceptual):
 - * Only the album owner may edit or delete it.
 - * Read access permitted if `is_public = True`.

PhotoViewSet

- Handles file uploads via the `image` field.
- DRF automatically parses `multipart/form-data`.
- Permissions:
 - Only the owner may update or delete photos.
 - Read access allowed if the related album is public.

ShareViewSet

- **list**: Returns shares involving the authenticated user (e.g., `Share.objects.filter(shared_with=request.user)`).
- **retrieve**: Fetches a share via its token (configured as the lookup field).
- **create**: Uses `ShareSerializer` to validate input, generate the UUID token, and save the share instance.

Custom Action: `redeem` Defined using:

```
@action(detail=True, methods=['post'])
```

Behavior:

- Marks `redeemed = True`.
- If `can_edit = True`, may add the requesting user to the album/photo's permission set (implementation-specific).
- All actions depend on the authenticated user as extracted from the JWT.

2.6 How Components Interact (Runtime Flow)

2.6.1 General Request Flow

1. **Client → API** The frontend sends an HTTP request with a JWT included in the `Authorization: Bearer <token>` header.
2. **Django Middleware** The JWT authentication middleware extracts the token, validates its signature, and sets `request.user` to the authenticated user instance.
3. **URL Resolution** Django's URL dispatcher matches the path (e.g., `/api/gallery/albums/`) to a corresponding view or viewset method (e.g., `AlbumViewSet.list`).
4. **View/ViewSet Execution** The view obtains a queryset appropriate for the action. Examples:
 - `Album.objects.filter(owner=request.user)` for owner-restricted operations.
 - `Photo.objects.all()` for publicly viewable resources (subject to permission checks).
5. **Serializer Processing** Serializers convert model instances to JSON or validate incoming JSON payloads. Custom logic may run, such as:
 - `has_unseen_edits` (method field).
 - Resolving a username string into a `User` object.
6. **Model Layer / ORM** Django ORM handles database queries. Foreign-key relationships are automatically resolved, e.g.,

```
photo.owner    # returns User instance
```
7. **Response Rendering** DRF renders the serializer output into JSON and returns it to the client.

2.6.2 Media File Handling

When a `Photo` instance is created:

- The uploaded file is stored under:

```
MEDIA_ROOT/photos/<date>/
```
- The API returns the public media URL such as:

```
/media/photos/<date>/<filename>
```
- The frontend can directly display the image using the returned URL.

2.6.3 Share Flow

1. Share Creation

1. The creator sends:

```
POST /api/gallery/shares/  
{  
    "album": <uuid>,  
    "username": <target_user>  
}
```

2. `ShareSerializer` validates:
 - XOR rule (album xor photo must be provided).
 - Provided username corresponds to an existing `User`.
3. A new `Share` row is stored with:
 - **token**: auto-generated UUID.
 - **shared_with**: resolved user.
4. The token is returned to the creator.

2. Accessing Shared Content

1. The recipient navigates to:

```
/api/gallery/shares/<token>/
```

2. The view resolves the share via token lookup, verifies access permissions, checks whether the share was already redeemed, and returns a structured payload using `SharedContentSerializer`.

3. Redeeming a Share

If the recipient performs the `redeem` action:

1. The share is marked:
`redeemed = True`
2. If `can_edit = True`, the system may (implementation-specific):
 - Grant the user edit permissions on the album or photo.
3. The response confirms successful redemption.

2.7 Security & Production Checklist

Concern	Recommended Setting
Secret Management	Move <code>SECRET_KEY</code> , Google OAuth credentials, and database passwords into environment variables or a secret manager.
CORS	Replace: <code>CORS_ALLOW_ALL_ORIGINS = True</code> with <code>CORS_ALLOWED_ORIGINS = ['https://myfrontend.com']</code> to restrict cross-origin access.
HTTPS	Serve Django behind an HTTPS-terminating reverse proxy (Nginx, Traefik) to ensure encrypted transport and secure handling of JWTs.
Token Lifetime	Configure: <code>SIMPLE_JWT = {ACCESS_TOKEN_LIFETIME: 15 min, REFRESH_TOKEN_LIFETIME: 7 days}</code> . Short-lived access tokens reduce security risk.
Database	Use PostgreSQL in production. Update <code>DATABASES</code> and run migrations on the production DB.
Media Storage	Use S3 (via <code>django-storages</code>) and set <code>DEFAULT_FILE_STORAGE = 'storages.backends.s3boto3.S3Boto3Storage'</code> .
Rate Limiting	Enable DRF throttling (e.g., <code>UserRateThrottle</code>) to prevent brute-force attacks.
Logging	Configure Django logging to record authentication failures, share creation, token exchange operations, and unexpected errors.
Input Validation	Ensure serializers validate all data. Use <code>IsAuthenticated</code> , <code>IsOwner</code> , and custom permission classes.
Testing	Write automated tests for serializers, viewsets, permissions, and the Google OAuth flow. Run them in CI/CD before deployment.

3 Frontend Implementation

3.1 Project Architecture

3.1.1 Technology Stack

- **Core Framework:** React 18 (Vite)
- **Language:** JavaScript (ES6+)
- **Routing:** React Router DOM v6
- **State Management:** React Context API
- **Styling:** Tailwind CSS
- **HTTP Client:** Axios
- **Icons:** Lucide React

Key Libraries

- `react-image-crop`: Image cropping functionality.
- `jwt-decode`: Parses JWT tokens to extract user metadata.
- `lucide-react`: Provides consistent SVG icons.

3.1.2 Directory Structure

```
frontend/src/
  api/
    client.js          # Configured Axios instance (BaseURL + Interceptors)
  assets/              # Static assets (images, fonts)
  components/         # Reusable UI Components
    AccessManagement.jsx # (Not currently used in main flow)
    ErrorBoundary.jsx   # Error catching wrapper
    Layout.jsx          # Main application shell (Navbar + Content)
    ProfileMenu.jsx     # Dropdown menu for user actions
    ShareModal.jsx      # Modal logic for creating share links
  context/
    AuthContext.jsx     # Global Authentication State Provider
  pages/              # Page-level Components (Views)
    AlbumDetail.jsx     # View single album contents
    Dashboard.jsx       # User's homepage (Albums & Shared)
    Editor.jsx          # Core Image Editor
    GoogleCallback.jsx  # OAuth redirect handler
    Landing.jsx         # Public entry page
    Login.jsx           # Sign in form
    Register.jsx        # Sign up form
    Search.jsx          # Search results page
    Share.jsx           # Public/Private share view handler
    Upload.jsx          # File upload interface
  utils/
    canvasUtils.js      # Helper functions for image manipulation
  App.jsx              # Route definitions and Provider wrapping
  main.jsx             # Application entry point
  index.css            # Global styles & Tailwind directives
```

3.2 Authentication System (`AuthContext.jsx`)

The application uses JWT (JSON Web Token)-based authentication, with global state managed through `AuthContext`. It centralizes user session handling, token persistence, and authentication-aware HTTP communication.

3.2.1 State Variables

- **user (Object—null)**: The authenticated user profile.
- **token (String—null)**: The JWT access token.
- **loading (Boolean)**: Indicates whether the app is checking for an existing stored token.

3.2.2 Key Functions

`login(username, password)` Sends a POST request to `/auth/login/`. On success, stores the access and refresh tokens in `localStorage`, updates context state, and sets the default Authorization header for Axios.

`register(username, email, password)` Sends a POST request to `/auth/register/` to create a new account.

`logout()` Clears `localStorage`, removes the Authorization header, resets user and token state, and redirects back to the homepage.

`useEffect (Initialization)` Executes when the application loads. If a JWT token exists in `localStorage`, `AuthContext`:

1. sets the Authorization header, and
2. fetches the authenticated user's profile from `/auth/profile/`.

3.2.3 Axios Interceptor (`api/client.js`)

The Axios client instance is preconfigured with a base URL and dynamically updated headers based on `AuthContext` state.

```
// On Login / Initialization:
client.defaults.headers.common['Authorization'] = 'Bearer ${token}';

// On Logout:
delete client.defaults.headers.common['Authorization'];
```

3.3 Core Pages & Implementation Details

3.3.1 Dashboard (`pages/Dashboard.jsx`)

The Dashboard serves as the central hub for authenticated users, providing access to their albums and shared content. It maintains several pieces of state, including the list of albums owned by the user, items shared with them, and UI toggles for modal visibility. The component also tracks the album currently being edited.

State Variables

- **albums (Array):** Albums owned by the authenticated user.
- **sharedContent (Array):** Items shared with the user.
- **isCreateOpen, isRedeemOpen (Boolean):** Modal visibility toggles.
- **editingAlbum (Object—null):** The album currently being edited.

Key Logic

- **Fetching:** On mount, the component requests `/gallery/albums/` and `/gallery/shares/receive`
- **Filtering:** The `fetchAlbums` function filters results to ensure only albums where `album.owner === user.id` are shown.
- **Redemption Logic:** The `handleRedeem` function parses user input; if a full URL is pasted, it extracts the token by splitting on `/share/`.

3.3.2 Photo Editor (pages/Editor.jsx)

The Photo Editor is the most complex component, responsible for all image manipulation and rendering tasks. It manages multiple layers of state related to filters, cropping, drawing, text overlays, and saved history.

State Breakdown

- **imageSrc (String/BlobUrl):** The displayed image source.
- **history (Array):** Stack of previous image states used for undo functionality.
- **activeTab (String):** Indicates the active tool (adjust, crop, doodle, text).
- **adjustments (Object):** Stores values for brightness, contrast, saturation, and blur.
- **crop, completedCrop, rotation:** Control cropping behavior and rotations from `react-image-crop`.
- **doodles (Array):** Drawing paths consisting of coordinates, color, and brush size.
- **textBoxes (Array):** Text overlays with position and content.

Tool Logic

- **Filters:** CSS filters are applied for preview. When finalized, `canvasUtils.applyFilters` renders the filtered version on an off-screen canvas and produces a new Blob.
- **Crop:** `react-image-crop` provides crop coordinates in percentages. The `applyCrop` function converts them into pixel-based coordinates using the image's natural dimensions.
- **Doodle:** An SVG overlay captures mouse movement for drawing. The `applyDoodle` function renders these paths on a canvas.
- **Save Actions:**
 - **Overwrite:** Uses a PATCH request to `/gallery/photos/${id}/`.
 - **Save Copy:** Uses a POST request to `/gallery/photos/`.

All save operations send `multipart/form-data` to properly upload files.

3.3.3 Sharing (pages/Share.jsx)

This component processes share links and renders shared content accordingly. It is accessed via the route `/share/:token` and determines whether the user has view or edit permissions.

Logic Overview

- Fetches shared data from `/gallery/shares/view/${token}/`.
- If the backend returns **401 Unauthorized**, it redirects the user to `/login`, preserving the redirect path for after authentication.
- If `response.data.can_edit` is true, a `shareToken` value is stored in `localStorage` for future operations (though this may be redundant if the token remains in the URL).
- The component conditionally renders either a full album or single photo view depending on the returned `type`.

3.3.4 Google Auth (pages/GoogleCallback.jsx)

This page completes the Google OAuth login process after the user is redirected back from Google's authorization server.

Logic Flow

1. Extracts the `code` parameter from the URL.
2. Sends the code to the backend endpoint `/auth/google/`.
3. On success, stores the received `access_token` and `refresh_token` and redirects the user to `/dashboard`.
4. A `useRef` guard ensures the component logic does not run twice in React Strict Mode.

4 Cloud Deployment and Infrastructure

4.1 Overview

The application is deployed on Amazon Web Services (AWS) using a monolithic containerized architecture. The system leverages **Amazon EC2** for compute, **Amazon RDS** for persistent relational data storage, and **Amazon S3** for scalable media object storage. The application is containerized using Docker and orchestrated via Docker Compose.

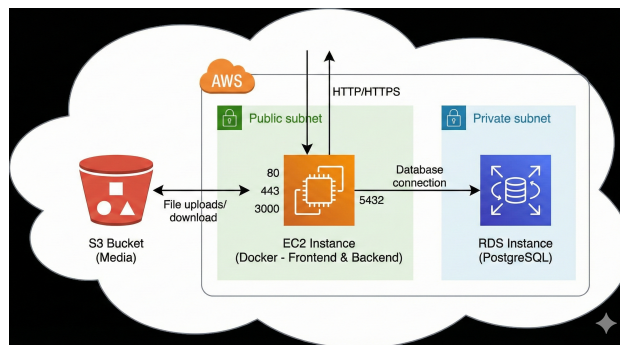


Figure 2: AWS 3-Tier Architecture with EC2, RDS, and S3

4.2 Infrastructure Provisioning

4.2.1 Database Layer (Amazon RDS)

A PostgreSQL instance was provisioned to handle user data and metadata.

- **Service:** Amazon RDS (Relational Database Service)
- **Engine:** PostgreSQL (Free Tier eligible)
- **Instance Identifier:** `photo-app-db`
- **Network Security:** Placed within a VPC Security Group (`rds_sg`) configured to allow inbound traffic on port `5432` exclusively from the application server (EC2) security group.
- **Access Control:** Public access is disabled to ensure database isolation.

4.2.2 Object Storage Layer (Amazon S3)

An S3 bucket is utilized for storing user-uploaded media files (JPEG, PNG).

- **Bucket Name:** `my-photo-app-media-[unique-id]`
- **Access Policy:** "Block All Public Access" was disabled to allow direct media retrieval. A specific Bucket Policy was applied to grant public read access to objects:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadGetObject",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::YOUR_BUCKET_NAME/*"
    }
  ]
}
```

- **CORS Configuration:** Cross-Origin Resource Sharing (CORS) was configured to allow GET, PUT, and POST requests from the frontend application.

4.2.3 Compute Layer (Amazon EC2)

The application backend and frontend run on a virtualized Linux server.

- **AMI:** Ubuntu Server 24.04 LTS
- **Instance Type:** t2.micro
- **IAM Role:** An IAM role with `AmazonS3FullAccess` policy was attached to the instance, enabling secure, keyless authentication between the backend and S3.
- **Security Group (ec2-sg):**
 - **SSH (22):** Restricted to administrator IP.
 - **HTTP (80) & HTTPS (443):** Open to 0.0.0.0/0 (Anywhere).

4.3 Deployment Procedure

4.3.1 Containerization and Registry (Local Environment)

The application services were built into Docker images and pushed to a central registry (Docker Hub) to facilitate deployment.

Commands Executed:

```
# Authenticate with Registry
docker login

# Build & Push Backend
docker build -t youruser/photo-backend ./backend
docker push youruser/photo-backend

# Build & Push Frontend
docker build -t youruser/photo-frontend ./frontend
docker push youruser/photo-frontend
```

4.3.2 Server Configuration

The EC2 instance was initialized with the necessary container runtime environment.

1. **Remote Access:** Established via SSH using the private key pair (.pem).
2. **Dependency Installation:**

```
sudo apt update
sudo apt install docker.io docker-compose -y
sudo usermod -aG docker ubuntu
newgrp docker
```

4.3.3 Orchestration Setup

A docker-compose.prod.yml file was created on the server to define the production services.

Configuration:

```
version: '3.8'
services:
  backend:
    image: youruser/photo-backend:latest
    ports:
      - "8000:8000"
    environment:
      - DEBUG=False
      - SECRET_KEY=${PRODUCTION_SECRET_KEY}
      - DATABASE_URL=postgres://postgres:${DB_PASSWORD}@${RDS_ENDPOINT}:5432/postgres
      - ALLOWED_HOSTS=${EC2_PUBLIC_IP}
      - USE_S3=True
      - AWS_STORAGE_BUCKET_NAME=${S3_BUCKET_NAME}
      - AWS_S3_REGION_NAME=us-east-1

  frontend:
    image: youruser/photo-frontend:latest
    ports:
      - "80:80"
    depends_on:
      - backend
```

4.3.4 Execution and Verification

The application stack was launched in detached mode, and database migrations were applied to the live RDS instance.

Launch Commands:

```
# Start Services
docker-compose -f docker-compose.prod.yml up -d

# Apply Database Schema
docker-compose -f docker-compose.prod.yml exec backend python manage.py migrate
```

4.4 Future Enhancements

For a fully production-ready environment, the following improvements are recommended:

- **SSL/TLS:** Implementation of HTTPS using AWS Certificate Manager (ACM).
- **Load Balancing:** Deployment of an Application Load Balancer (ALB) to manage traffic distribution.
- **DNS:** Configuration of a custom domain via Amazon Route 53.

5 Future Improvements

- Disallow Photos/Albums with the same name.
- Download full albums and not just individual pictures
- Allow users to change their profile picture, password and usernames
- Enable sharing with email instead of username
- Enhance real time collaboration with real-time comments
- Only the primary owner determines access to shared photos/albums

6 References

- AWS Documentation: <https://docs.aws.amazon.com/>
- Dockerization: <https://docs.docker.com/>
- General Information: <https://www.geeksforgeeks.org/>
- React Documentaion: <https://react.dev/learn>

7 Declarations

- **Roshni Pai:** No conflict declared. Work was split equally.
- **Poulomi Sarkar:** No conflict declared. Work was split equally.